
1. INTRODUCTION

- Equations of motion
- There needs to be computational support for modeling acoustic metamaterials
- Big problems demand high-performance solvers

2. METHODS

There are 3 traditional ways to parallelize the solution of a computational problem: CPU parallelization, GPU parallelization, and Distributed computing. While CPU parallelization is more straightforward to implement, GPU parallelization can allow for runtimes to decrease by many orders of magnitudes. Even lower run times can be achieved by combining either of these parallelization schemes with running them on multiple machines. This investigation focuses on parallelizing the solution of equations of motion using GPUs and multiple machines.

These approaches can offer massive increases in performance, but only for problems that are well-posed to be parallelized. Traditionally, initial value problems have been unable to be parallelized due their dependance on causality. Several methods have been created to overcome this limitation. These methods include the Parareal algorithm, Multigrid Reduction in Time (MGRIT), Parallel Full Approximation Scheme in Space and Time (PFASST). This investigation focuses on the Parareal algorithm.

A. THE PARAREAL ALGORITHM

The Parareal algorithm does not solve a problem directly, but deconstructs the problem in to subproblems that can be solved with traditional methods in parallel, then recombines their solutions to yield a solution that is equivalent to what would have been produced sequentially. The Parareal algorithm can be thought of as shooting-in-time due its predictor-corrector nature, or multi-grid-in-time due to its layered discretization in the time domain.

The Parareal algorithm can be broken down into four key steps

1. Prepare the subproblems
2. Solve each subproblem in parallel
3. Recombine and correct
4. Loop

i. Subproblem Preparation

If the root problem P can be described by a second-order differential equation $\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t)$, initial values $u(0) = u_0, \partial_t u(0) = v_0$, and time domain $D = [T, T + \Delta T]$, then define P as the initial value problem such that

$$P = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [T, T + \Delta T]\}. \quad (1)$$

Then choose a natural number N as the number of subproblems that root problem should be partitioned into; this should probably be the number of compute cores (either CPU or GPU).

Then partition the time domain for $p = 0, 1, \dots, N - 1$ so that

$$D_p = [T + p\Delta T/N, T + (p + 1)\Delta T/N] = [T_p, T_{p+1}] \quad (2)$$

Choose a coarse propagator $\mathcal{C}_0$, e.g. semi-implicit Euler or another algorithm that is cheap to evaluate, and propagate the initial values according to the differential equation. The result of this is

$$\{u_p^0\}_p = \{u_0^0, u_1^0, u_2^0, \dots, u_{N-1}^0\} \quad (3a)$$

$$\{v_p^0\}_p = \{v_0^0, v_1^0, v_2^0, \dots, v_{N-1}^0\} \quad (3b)$$

where the superscript denotes that this is the zeroth-iteration. Normally, this is where the story would end, but the Parareal algorithm is just getting started.

Now the subproblem P_p^i for partition p at iteration i is defined such that

$$P_p^i = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_p^i, \partial_t u(0) = v_p^i \quad D_p = [T_p, T_{p+1}]\} \quad (4)$$

The collection of subproblems at iteration i is then defined by $P^i = \{P_p^i\}_p$.

ii. Parallel Propagation

In addition to parallelizing, part of the magic of the Parareal algorithm lies in solving each subproblem not once, but twice with different solves or propagators. The next step in the process is to choose coarse, and fine. It should be noted that this coarse propagator does not need to be the same as the coarse propagator that was chosen in preparing the subproblems. Possible propagators include the semi-implicit Euler method with a large time step for the coarse propagator, and the velocity-verlet method with a small time step for the fine propagator; in order to satisfy energy-conservation, symplectic integrators should be used. Without loss of generality, let the chosen coarse and fine propagators be denoted $\mathcal{C}, \mathcal{F}$, respectively, and the n_S -th data point for propagator $\mathcal{S}$ in the p -th subproblem at iteration i be denoted $u_{pn_S}^i$ and defined traditionally by $u_{pn_S}^i = \mathcal{S}u_{p, n_S-1}^i$, and the solution from propagator $\mathcal{S}$ on subproblem p is the ordered collection of points $\mathcal{S}u_p^i = \{u_{pn_S}^i\}_{n_S}$.

iii. Corrections

- Coarse propagate big-picture solution but with each term corrected

iv. Iterate

B. GPU COMPUTING

C. DISTRIBUTED COMPUTING

3. RESULTS

4. DISCUSSION

5. CONCLUSION

A. FUTURE WORK

- Krylov Enhanced Subspaces
-